

DOCUMENTACION TECNICA

CARTA DIGITAL SAAS - PLATAFORMA QR

Estructura, Arquitectura, Esquemas y Pseudocodigo

Plataforma:	Next.js 15, TypeScript, Supabase, Material UI
Base de Datos:	PostgreSQL (Supabase DB con RLS habilitado)
Version:	1.2.0 (Rediseño de Carta y Sistema de Resenias)
Fecha de Emision:	Julio, 2026

1. Arquitectura del Proyecto

Estructura de Carpetas

El proyecto esta estructurado como una aplicacion SaaS moderna usando Next.js 15 App Router. A continuacion se detallan las carpetas principales y su proposito:

- **src/actions:** Contiene las Acciones del Servidor (Server Actions), encargadas de interactuar de forma segura con la base de datos Supabase, ejecutar operaciones CRUD y gestionar la autentificacion.
- **src/app:** Define las rutas de la aplicacion. Utiliza subcarpetas dinamicas (como `c/[slug]`) y carpetas de grupo de rutas (como `(auth)` y `(dashboard)`) para modularizar el proyecto.
- **src/components:** Componentes modulares reutilizables. Separados en `dashboard` (administracion), `ui` (componentes basicos de control) y `public` (la interfaz publica de la carta del restaurante).
- **src/lib:** Configuraciones de cliente y servidor de Supabase, asi como middlewares de autentificacion.
- **src/types:** Definiciones de tipos de TypeScript correspondientes a las entidades de negocios, productos, categorias y reviews.

Detalle de Archivos del Sistema

- **src/middleware.ts:** Intercepta las peticiones HTTP entrantes para validar la sesion del usuario y redirigir segun corresponda.
- **src/lib/supabase/server.ts:** Inicializador del cliente Supabase del lado del servidor para Server Actions y Server Components.
- **src/lib/supabase/client.ts:** Inicializador del cliente Supabase del lado del navegador para la interaccion del usuario.
- **src/lib/supabase/middleware.ts:** Gestiona las cookies de sesion y las actualiza durante las transiciones de rutas protegidas.

2. Esquema de Base de Datos

La base de datos esta alojada en Supabase y utiliza PostgreSQL. Tiene habilitado el control de acceso basado en roles y politicas de seguridad a nivel de fila (Row Level Security - RLS) para aislar los datos de los diferentes comercios.

Estructura de las Tablas principales

Tabla: profiles

Almacena la informacion del perfil del administrador.

- **id (UUID, PK):** Identificador unico referenciado de auth.users (Supabase Auth).
- **name / email (TEXT):** Nombre completo y direccion de correo del administrador.
- **created_at (TIMESTAMPTZ):** Fecha y hora de creacion.

Tabla: businesses

Almacena la configuracion y personalizacion visual de cada comercio.

- **id (UUID, PK):** Identificador unico autogenerado.
- **owner_id (UUID, FK):** Referencia al creador (profiles.id).
- **name / slug (TEXT):** Nombre comercial y el identificador unico URL (ej: bella-vista).
- **logo_url / cover_image / banner_image (TEXT):** Direcciones URL de imagenes almacenadas en el Storage.
- **color_primary / color_secondary (TEXT):** Colores hexadecimales de la identidad visual de la marca.
- **typography / theme (TEXT):** Tipografia seleccionada y tema general (light o dark).

Tabla: categories

Agrupar los productos de un comercio.

- **id (UUID, PK):** Identificador unico autogenerado.
- **business_id (UUID, FK):** Referencia al negocio (businesses.id).
- **name (TEXT):** Nombre de la categoria (ej: Entradas, Bebidas).
- **item_order (INTEGER):** Orden secuencial de aparicion.

Esquema de Base de Datos (Cont.)

Tabla: products

Almacena los productos de la carta digital.

- **id (UUID, PK)**: Identificador unico autogenerado.
- **business_id (UUID, FK)**: Referencia al negocio propietario (businesses.id).
- **category_id (UUID, FK)**: Referencia a la categoria del producto (categories.id).
- **name / description (TEXT)**: Nombre y descripcion del plato o bebida.
- **price (NUMERIC)**: Precio unitario con decimales.
- **image_url (TEXT)**: Direccion URL de la imagen del producto.
- **is_available (BOOLEAN)**: Filtro para mostrar u ocultar productos no disponibles en la carta.

Tabla: reviews

Registra las opiniones y calificaciones aportadas de forma publica por los clientes.

- **id (UUID, PK)**: Identificador unico autogenerado.
- **business_id (UUID, FK)**: Referencia al negocio evaluado (businesses.id).
- **first_name / last_name (TEXT)**: Nombre y apellido del autor del comentario.
- **comment (TEXT)**: Texto de la reseña compartida.
- **rating (INTEGER)**: Calificacion numerica por estrellas (del 1 al 5).

Tabla: settings

Configuraciones operativas del negocio (moneda, redes sociales, direccion, whatsapp).

Políticas RLS e Indices implementados

Políticas de Lectura Publica: Permitidas para businesses, categories, products, reviews y settings (utilizando true) para permitir el acceso publico sin autenticacion a la carta digital.

Políticas de Escritura/Modificacion: Restringidas mediante cheques de seguridad (auth.uid() = owner_id), asegurando que solo el propietario del comercio pueda modificar sus productos, categorias y configuraciones.

Indices en Reviews: Se agregaron indices en business_id y created_at para optimizar el ordenamiento temporal y filtrado por negocio.

3. Acciones de Servidor (Server Actions)

Las Server Actions encapsulan la lógica de negocio en el servidor y son llamadas directamente desde los componentes cliente.

Pseudocódigo de Acciones de Resenias (reviews.ts)

Funcion: **getBusinessBySlug**

```
GET_BUSINESS_BY_SLUG(slug: String):
  supabase = CreateSupabaseServerClient()
  response = supabase.FROM("businesses")
    .SELECT("*")
    .WHERE("slug" == slug)
    .SINGLE()

  IF response.error:
    RETURN { error: response.error.message }
  RETURN { data: response.data as Business }
```

Funcion: **getProducts**

```
GET_PRODUCTS_FOR_PUBLIC_PAGE(businessId: UUID):
  supabase = CreateSupabaseServerClient()
  response = supabase.FROM("products")
    .SELECT("*, category:categories(id, name)")
    .WHERE("business_id" == businessId)
    .WHERE("is_available" == true)
    .ORDER_BY("item_order" ASC)

  IF response.error:
    RETURN { error: response.error.message }
  RETURN { data: response.data as Product[] }
```

Funcion: **searchProducts**

```
SEARCH_PRODUCTS(businessId: UUID, query: String):
  supabase = CreateSupabaseServerClient()

  # Buscar categorias que coincidan con la consulta
  categories = supabase.FROM("categories")
    .SELECT("id")
    .WHERE("business_id" == businessId)
    .WHERE("name" LIKE "%" + query + "%")
```

```
# Construir consulta dinamica OR
dbQuery = supabase.FROM("products")
                .SELECT("*, category:categories(id, name)")
                .WHERE("business_id" == businessId)
                .WHERE("is_available" == true)

IF query NOT EMPTY:
    dbQuery.OR("name LIKE %query% OR description LIKE %query% OR category_id IN
(categories)")

response = dbQuery.ORDER_BY("item_order" ASC)
RETURN { data: response.data }
```

Acciones de Servidor (Cont.)

Funcion: getReviews

```
GET_REVIEWS(businessId: UUID):
  supabase = CreateSupabaseServerClient()
  response = supabase.FROM("reviews")
    .SELECT("*")
    .WHERE("business_id" == businessId)
    .ORDER_BY("created_at" DESC)
  RETURN { data: response.data as Review[] }
```

Funcion: createReview

```
CREATE_REVIEW(businessId: UUID, reviewData):
  supabase = CreateSupabaseServerClient()

  # Validar datos en el servidor
  IF reviewData.first_name EMPTY OR reviewData.last_name EMPTY OR reviewData.comment EMPTY:
    RETURN { error: "Todos los campos son requeridos" }

  # Insertar en base de datos
  insertResponse = supabase.FROM("reviews")
    .INSERT({
      business_id: businessId,
      first_name: reviewData.first_name,
      last_name: reviewData.last_name,
      comment: reviewData.comment,
      rating: reviewData.rating
    })

  # Obtener el slug para revalidar la cache de Next.js
  business = supabase.FROM("businesses")
    .SELECT("slug")
    .WHERE("id" == businessId)
    .SINGLE()

  IF business:
    REVALIDATE_PATH("/c/" + business.slug)

  RETURN { data: insertResponse.data }
```

Funcion: getAverageRating

```
GET_AVERAGE_RATING(businessId: UUID):
  supabase = CreateSupabaseServerClient()
```

```
reviews = supabase.FROM("reviews")
                .SELECT("rating")
                .WHERE("business_id" == businessId)

totalReviews = LENGTH(reviews)
IF totalReviews == 0:
    RETURN { data: { average: 0, count: 0 } }

sumRating = SUM(r.rating FOR EACH r IN reviews)
average = ROUND(sumRating / totalReviews, 1)

RETURN { data: { average: average, count: totalReviews } }
```


4. Ruta Pública de Carta (/c/[slug])

Funcionamiento y Renderizado Dinámico

La página pública (/c/[slug]) es el núcleo del sistema. Se encarga de procesar el slug introducido en la URL, cargar las dependencias de Supabase correspondientes al comercio de forma segura, y adaptar la interfaz del usuario con los estilos inline específicos sin requerir de hojas de estilo fijas.

Flujo de Carga de la Página

- **1. Resolución del Comercio:** Se recupera el slug desde los parámetros de la ruta. Como Next.js 15 utiliza promesas asincrónicas para "params", se realiza "await params" antes de invocar la acción "getBusinessBySlug(slug)".
- **2. Verificación de Existencia:** Si el comercio no existe en la base de datos o el slug no es correcto, se dispara de inmediato la función "notFound()" de Next.js.
- **3. Consulta Concurrente:** Se consultan los productos disponibles (donde is_available = true) y la lista de reviews asociadas al identificador del comercio de forma simultánea.
- **4. Inyección Dinámica de Estilos:** Se genera una etiqueta de estilo <style> dinámica en el cuerpo del DOM que define variables CSS globales (--primary-color, --secondary-color, --font-family).
- **5. Fuentes Google Fonts en Caliente:** Se importa dinámicamente la fuente configurada (Inter, Roboto, Playfair Display, Montserrat, etc.) desde Google Fonts mediante una etiqueta <link> construida en tiempo de ejecución.

Código de Inyección de Variables CSS

```
# Dentro de src/app/c/[slug]/page.tsx
primaryColor = business.color_primary OR "#f97316"
secondaryColor = business.color_secondary OR "#ffffff"
fontName = business.typography OR "Inter"

STYLE_TAG = ""
:root {
  --primary-color: {primaryColor};
  --primary-color-rgb: {hexToRgb(primaryColor)};
  --secondary-color: {secondaryColor};
  --font-family: '{fontName}', sans-serif;
  --bg-page: #0a0e1a;
  --bg-card: #111827;
  --border-color: #1e2d45;
}
body {
  font-family: var(--font-family);
  background-color: var(--bg-page);
```

```
}  
"" ""
```

5. Componentes de la Carta Publica

Los componentes de la carta publica fueron disenados de forma aislada en la carpeta `src/components/public/` para no alterar las dependencias del area administrativa. Estan optimizados usando inline styles y TailwindCSS.

Descripcion de Componentes Publicos

Navbar.tsx

Barra de navegacion flotante/fija con efecto blur translucido (`backdrop-filter`) que muestra el logotipo del comercio, titulo y enlaces con scroll suave (`#menu`, `#about`, `#reviews`). Dispone de un Drawer lateral de Material UI para dispositivos moviles.

SearchBar.tsx

Caja de busqueda premium con bordes que responden dinamicamente al color de foco principal del negocio. Incluye un boton interactivo para limpiar el texto introducido.

MenuSection.tsx

Controla el estado de busqueda en tiempo real. Realiza la busqueda de forma reactiva en el cliente filtrando los productos cargados por Nombre, Descripcion y Categoria, logrando una experiencia instantanea.

ProductGrid.tsx

Layout flexible responsivo que utiliza una distribucion Masonry basada en CSS (`column-count`). Esto permite ajustar la altura exacta de las tarjetas de platos sin dejar espacios vacios verticales.

ProductCard.tsx

Tarjeta premium de producto. Muestra la imagen (con `object-cover`), el titulo, la descripcion y el precio a la derecha utilizando el color primario del negocio. Al pasar el cursor, genera una transicion de escala y un sombreado suave.

AboutSection.tsx

Seccion moderna con estructura asimetrica de rejilla. Renderiza el titulo e historia detallada del restaurante junto a la imagen principal de presentacion comercial del establecimiento.

ReviewSection.tsx

Contenedor del modulo de comentarios. Conecta el panel de OverallRating con el listado de ReviewCard. Controla la apertura del modal y actualiza la lista de opiniones tras insertar una reseña sin recargar la pagina.

ReviewDialog.tsx

Formulario Dialog de Material UI con validaciones completas. Permite calificar usando la interfaz interactiva de StarRating y enviar los datos de la reseña a Supabase.

StarRating.tsx

Componente flexible. Actua como selector interactivo (utilizando estados de hover reactivos) o visualizador estatico de estrellas evaluadas.